

AgroScan AI

A complete walkthrough of the model training pipeline and the full-stack web application – architecture, design decisions, key concepts, and likely interview questions with model answers.

YOLO11 Classification

Transfer Learning

Flask + SQLite

PlantVillage Dataset

Computer Vision

Full-Stack ML

Contents

- 01** The One-Line Pitch & Elevator Explanation
- 02** System Architecture — How The Two Halves Fit Together
- 03** Full Technology Stack & Rationale
- 04** The Training Pipeline, Explained Step by Step
- 05** The Web Application, Component by Component
- 06** Design Decisions Worth Defending
- 07** Known Limitations
- 08** Future Improvements
- 09** Glossary of Key Terms
- 10** Likely Interview Questions & Model Answers
- 11** Final Cheat Sheet (One-Page Recap)

The One-Line Pitch & Elevator Explanation

"AgroScan AI is a full-stack plant disease detection system: a YOLO11 image classification model, fine-tuned on the PlantVillage dataset across 38 crop/disease classes, served through a Flask web application with user accounts, a 'gatekeeper' model that rejects non-plant images, per-user scan history, an analytics dashboard, and multi-language support for regional Indian languages."

That single sentence is your 15-second answer to "tell me about a project you've built." But interviewers will almost always follow up, so it helps to have a slightly longer, two-minute version ready that naturally opens the door to deeper technical questions:

TWO-MINUTE VERSION

"The problem I was solving is that farmers often can't quickly identify what's wrong with a sick-looking crop, and by the time they get expert advice the disease may have spread. So I built an end-to-end system: I trained a lightweight image classifier on the PlantVillage dataset — about 54,000 labeled leaf images across 38 disease/healthy classes covering 14 crop species — using transfer learning on top of a YOLO11 nano backbone. Then I wrapped that model in a Flask web app where a user can register, upload a photo of a leaf, and get back a diagnosis with a confidence score and recommended remedies. Because a raw classifier will confidently mislabel literally anything you show it — even a photo of a car — I added a second, general-purpose model in front of it that acts as a sanity check, rejecting images that don't look plant-like at all. The app also keeps a history of each user's scans, shows trends on a dashboard, exports PDF reports, and translates itself into the farmer's preferred regional language."

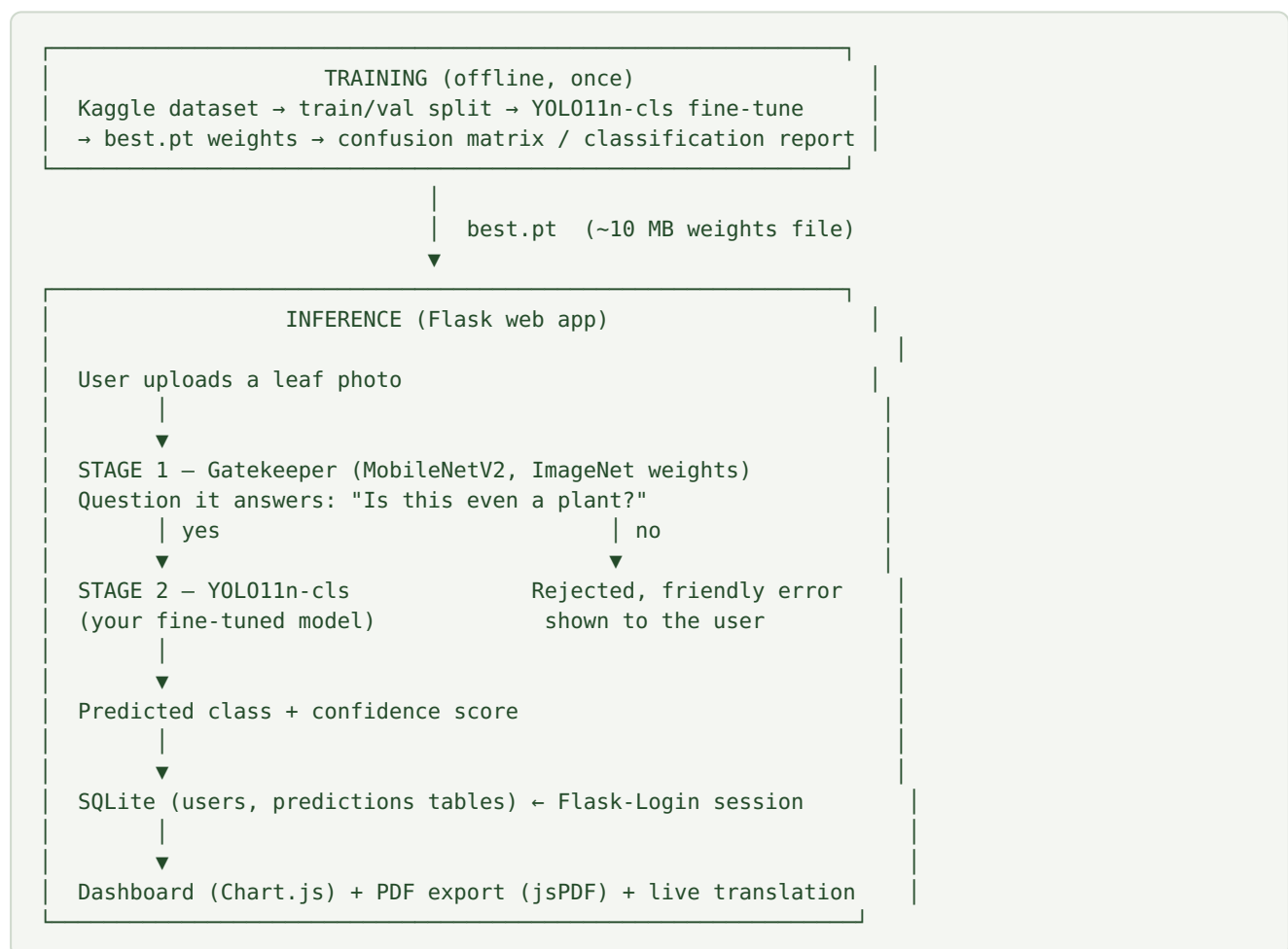
Notice that version does three things a good project summary should always do: states the **real-world problem**, explains the **technical approach** at a level a non-specialist could follow, and mentions one **non-obvious design decision** (the gatekeeper model) that signals you thought beyond just "train a model and call it done."

Two halves to know cold for the rest of this document:

1. **The training pipeline** (a Colab/Kaggle script) — how the disease-classifier model was actually built, from raw dataset to a deployable weights file.
2. **The web application** (a Flask project) — how that trained model gets served to real users, with everything a production-facing tool needs around it: auth, storage, UI, analytics, and safety nets.

System Architecture

At the highest level, the system has two phases that run at very different times and on very different infrastructure: training happens *once* (or occasionally, when you want to retrain on more data), offline, on a GPU. Inference happens *continuously*, online, on whatever server hosts the Flask app — and for this project, that server doesn't even need a GPU, because the nano model is light enough to run acceptably on CPU.



Why two models instead of one? (This is the single best design-decision question to prepare for.)

A single 38-class classifier trained only on plant leaf images has, mathematically, **no concept of "not a plant."** Classification models almost always end in a softmax layer, which by construction forces its outputs to sum to exactly 1 across the known classes. If you feed such a model a

photograph of a car, a shoe, or a person's face, it does not have the option to say "none of the above" — it will still distribute its confidence across the 38 classes it knows and report whichever one scores highest, sometimes with startlingly high confidence, simply because the softmax has to add up to 100% no matter what.

This is a well-known failure mode in machine learning called an **out-of-distribution (OOD) input** — data that doesn't resemble anything the model saw during training. Properly solving OOD detection is itself a research problem (energy-based scoring, Mahalanobis distance methods, dedicated "reject" classes trained on huge negative datasets, etc.), and building a robust custom solution would have been a significant project on its own.

Instead, this project takes a pragmatic shortcut: reuse an existing, already-trained general-purpose model (MobileNetV2, pretrained on the 1,000-class ImageNet dataset) purely as a sanity filter. ImageNet already includes plenty of leaf-adjacent, plant-adjacent, and food-adjacent categories, so if MobileNetV2's own top-5 predictions for an image don't contain any plant-related keyword (leaf, tree, vegetable, fruit, crop, etc.), that's a strong, cheap signal the image probably isn't a plant at all — and the specialist model never even needs to run.

HOW TO PHRASE THIS IF ASKED DIRECTLY

"I used a cascade of two models rather than training one bigger model, because the problems they solve are fundamentally different in kind: one is fine-grained classification among 38 similar-looking plant diseases, and the other is coarse out-of-distribution rejection. Reusing an off-the-shelf ImageNet model for the second problem was far cheaper than collecting a huge, diverse 'not a plant' dataset and training a dedicated detector, and it's a pattern — cascading a cheap general filter in front of an expensive specialist model — that shows up a lot in production ML systems for exactly this cost/accuracy tradeoff reason."

Full Technology Stack & Rationale

Being able to justify *every* item in your stack — not just describe it — is what separates "I followed a tutorial" from "I made engineering decisions." Here is the full stack with the reasoning behind each choice.

Layer	Technology	Why this, specifically
Classification model	YOLO11n-cls (Ultralytics)	The "nano" variant trades some accuracy for a small, fast model that can run on a CPU in production without a dedicated GPU server — an appropriate tradeoff for a demo-scale deployment. The "-cls" suffix matters: Ultralytics ships the same YOLO11 backbone in detection, segmentation, and classification variants, and this task only needs whole-image classification, not bounding boxes.
Gatekeeper model	MobileNetV2 (Keras, ImageNet weights)	Lightweight, widely available, needs zero additional training — used purely for its general-purpose feature space and label vocabulary, not fine-tuned at all.
Training dataset	PlantVillage (via Kaggle)	The standard, most widely cited benchmark dataset for plant disease classification research — choosing it makes results comparable to published baselines.
Backend framework	Flask + Flask-Login	Minimal, unopinionated — appropriate for a small-to-medium app where a heavier framework like Django would add unused complexity (admin panel, ORM migrations, etc. this project doesn't need).
Database	SQLite	Zero-configuration, single-file, no separate database server to run — ideal for a single-instance demo. Explicitly a known limitation at real scale (see Section 07).
Frontend	Jinja2 templates + Tailwind CDN + Chart.js	Server-rendered HTML avoids a separate frontend build pipeline (no Webpack/Vite/npm build step to maintain) while Tailwind's utility classes keep styling fast to iterate on.

PDF export	jsPDF (pure client-side JavaScript)	Draws the report programmatically — rectangles, text, computed bar heights — directly from JSON data, rather than screenshotting the rendered dashboard DOM (the more common but fragile "html2canvas" approach). This avoids font-rendering inconsistencies and works even in restricted network/tunnel environments where a screenshot-based library might fail silently.
Internationalization	deep-translator (Google Translate backend)	Enables on-the-fly translation into 11 languages without maintaining hand-written locale files for every string — a deliberate tradeoff of translation quality for coverage speed.
Training environment	Google Colab + Kaggle API	Free GPU access (NVIDIA T4) removes the need for personal GPU hardware; the Kaggle API pulls the dataset directly into the Colab runtime without manual downloads.
Model export format	ONNX	A framework-agnostic format — once exported, the model could theoretically run outside PyTorch/Ultralytics entirely (e.g. in a browser via ONNX.js, on mobile, or behind a different serving stack), which is valuable optionality even if not exercised yet.
Production serving	Gunicorn (WSGI server)	Flask's built-in development server is explicitly documented as unsuitable for production traffic (single-threaded, no process management); Gunicorn provides proper worker management.

The Training Pipeline, Explained Step by Step

This is the section to walk an interviewer through if they ask "how did you actually train the model?" — go step by step through the script rather than jumping straight to the training call, since the setup and data preparation choices are just as revealing of your understanding as the `model.train()` line itself.

4.1 Environment Setup

The script installs its dependencies (`ultralalytics`, `kaggle`, `seaborn`, `matplotlib`, `scikit-learn`) at the very top, before importing anything that depends on them — a small but important ordering detail in a notebook environment, since Python will throw an `ImportError` if you try to import a package before it's installed in that runtime.

It then explicitly checks for a CUDA-capable GPU:

```
if not torch.cuda.is_available():  
    raise SystemError("STOP! You are on CPU. Go to Runtime > Change runtime type > T4 GPU")
```

This **hard-stops execution** rather than silently falling back to CPU. That's a deliberate choice: training even a small CNN classifier on CPU can take many hours instead of minutes, and failing fast with a clear instruction is far better than letting someone wait hours only to realize their runtime was misconfigured.

Kaggle credentials (`KAGGLE_USERNAME`, `KAGGLE_KEY`) are collected at runtime via `input()` and `getpass()` rather than hardcoded into the script. `getpass()` specifically hides the API key as it's typed, so it doesn't end up visible in the notebook's cell output or accidentally committed if the notebook is later shared — a small but genuinely good credential-hygiene habit worth mentioning if asked about security practices.

4.2 Data Acquisition & Preparation

The PlantVillage dataset is pulled directly from Kaggle (`abdallahalidev/plantvillage-dataset`) via the Kaggle CLI, then unzipped locally. The raw dataset ships already organized into per-class

folders (e.g. `Tomato__Early_blight/`, `Potato__healthy/`) — but critically, it ships with **no pre-made train/validation split**, so the script builds one itself:

- For every class folder independently, the images are shuffled and split **80% train / 20% validation**.
- Files are *moved* (not copied) into a new `plantvillage_data/train/<class>/` and `plantvillage_data/val/<class>/` directory structure.
- This exact folder layout — `train/<class_name>/*.jpg`, `val/<class_name>/*.jpg` — is precisely the format Ultralytics' classification trainer expects out of the box. No manual annotation file, no bounding-box labels, no dataset YAML config needed, which is one of the practical advantages of framing this as classification rather than object detection.

WHY SPLIT PER CLASS INSTEAD OF ONE GLOBAL SHUFFLE?

Splitting *within each class folder* keeps the split **stratified** — every class ends up represented in roughly the same 80/20 proportion in both train and validation sets. This matters a lot when classes are imbalanced in size (which is common in PlantVillage — some diseases have far more example photos than others). A single global shuffle across the whole dataset could, by bad luck, leave a rare class almost entirely in the training set and barely represented in validation, making your validation accuracy numbers misleading for that class.

4.3 Training

```
model = YOLO('yolo11n-cls.pt')
results = model.train(
    data='/content/plantvillage_data',
    epochs=15,
    imgsz=224,
    batch=64,
    workers=4,
    cache=True,
    amp=True,
    device=0
)
```

Every argument here is worth being able to explain individually — this is the part of the project most likely to get drilled into during a technical interview:

yolo11n-cls.pt — Starts from Ultralytics' own pretrained YOLO11 "nano" classification checkpoint, which was itself already trained on ImageNet. This is **transfer learning**, not training from scratch: the model already knows general, reusable visual features — edges, textures, color gradients, simple shapes — from having seen a huge, diverse dataset already. Fine-tuning only needs to adapt those existing features toward the much narrower, more

specific task of telling plant diseases apart, which requires far less data and far less training time than learning to see from nothing.

epochs=15 — One epoch is one complete pass through the entire training set. 15 is a fairly modest number, and that's appropriate specifically *because* this is fine-tuning a pretrained model rather than training from random initialization — a from-scratch model on a dataset this size would typically need many more epochs (and would likely still underperform this transfer-learning approach).

imgsz=224 — All images are resized to 224×224 pixels before being fed to the network. This is a very standard resolution for lightweight classifiers, chosen originally because it matches what most ImageNet-pretrained backbones (including MobileNet) were trained on — using the same resolution the pretrained weights expect avoids introducing a mismatch between pretraining and fine-tuning.

batch=64 — The number of images processed together in one forward/backward pass before the model's weights are updated. Larger batches make better use of GPU parallelism (more images processed simultaneously) and often produce more stable gradient estimates, at the cost of needing more GPU memory. 64 is a reasonable size for a T4 GPU with an image size of 224.

workers=4 — Number of parallel CPU threads/processes used just for loading and preprocessing images from disk. Without enough workers, the GPU can sit idle waiting for the next batch of images to be read and decoded — this setting keeps the data pipeline fed so the (much more expensive) GPU computation is the bottleneck, not disk I/O.

cache=True — After the first epoch, decoded images are cached in RAM rather than being re-read and re-decoded from disk on every subsequent epoch. This is a significant speed optimization whenever disk I/O — not GPU compute — turns out to be the limiting factor, which is common in cloud notebook environments like Colab where the "disk" is often network-backed storage.

amp=True — Enables **Automatic Mixed Precision**: the model performs the numerically safe parts of its computation in 16-bit floating point instead of the usual 32-bit, roughly halving memory usage and increasing training throughput on modern GPUs that have dedicated hardware for fast FP16 math (like the T4's Tensor Cores), while a small set of numerically sensitive operations stay in FP32 to avoid meaningfully hurting accuracy.

device=0 — Explicitly targets GPU index 0, i.e. the single GPU Colab assigns to the runtime.

4.4 Evaluation & Research Artifacts

After training finishes, the script doesn't just report Ultralytics' built-in summary metrics — it runs a full, independent evaluation pass to generate the kind of artifacts you'd expect in a research write-up or model card:

- **Training curves** — top-1 and top-5 accuracy over each epoch, plus train vs. validation loss, read straight from Ultralytics' `results.csv` and plotted with Matplotlib. These curves are the primary tool for visually diagnosing overfitting (see the glossary entry below).
- **Confusion matrix (38×38)** — built manually by re-running inference on every single validation image (processed in batches of 64 for efficiency) and comparing each prediction against its true label using `sklearn.metrics.confusion_matrix`, visualized as a Seaborn heatmap. This reveals *which specific classes get confused with which others* — for example, you'd generally expect "Tomato Early Blight" to be confused with "Tomato Late Blight" far more often than with "Apple Scab," since the first pair looks visually similar and the second doesn't.
- **Classification report** — precision, recall, and F1-score computed *per individual class*, via `sklearn.metrics.classification_report`. This matters more than a single overall accuracy number specifically because the dataset is imbalanced — a model could post a deceptively high overall accuracy by doing extremely well on a few common classes while quietly failing on rare ones, and per-class metrics are what expose that.
- **ONNX export** — `model.export(format='onnx')` converts the trained PyTorch model into ONNX's framework-agnostic format, opening the door to running it somewhere other than PyTorch/Ultralytics later (mobile apps, browser-based inference via ONNX.js, or a different serving stack entirely) without needing to retrain anything.
- Everything — the raw metrics CSV, both training curve plots, the confusion matrix image, the classification report CSV, and both exported model formats — gets bundled into a single `AgroScan_Research_Package.zip` and downloaded. This is effectively your "lab notebook" evidence folder, and exactly what you should open and review the night before an interview.

The Web Application, Component by Component

5.1 Authentication

`flask-login` manages session state — it's what makes `current_user` and the `@login_required` decorator work throughout the app, tracking who's logged in via a secure session cookie rather than requiring credentials on every single request.

Passwords are never stored in plaintext. `werkzeug.security.generate_password_hash` and `check_password_hash` handle this using salted password hashing under the hood — meaning even if the database were somehow leaked, an attacker would not directly recover anyone's actual password. ("Salting" means a random value is mixed into each password before hashing, so two users with the identical password still get completely different stored hashes, and precomputed "rainbow table" attacks become useless.)

The `users` table itself is intentionally minimal: `id`, `username (unique)`, `password_hash`.

5.2 Prediction Flow (`/predict`)

1. The user chooses an image source: a fresh upload, "reuse my last image," or a built-in sample leaf photo (useful for demoing the app without needing a real leaf on hand).
2. **Gatekeeper check** (`is_valid_plant`): the image is resized to 224×224 and run through MobileNetV2; its top-5 decoded ImageNet labels are checked against a hand-curated list of plant-related keywords (leaf, tree, fruit, crop, vegetable, etc.). If none match, the image is rejected with a friendly, specific error *before* the specialist model is ever invoked — saving compute and, more importantly, avoiding a confidently wrong diagnosis on an image that was never a plant to begin with.
3. **YOLO11 inference**: the fine-tuned classifier runs on the (now-validated) image, producing `probs.top1` (the index of the highest-scoring class) and `probs.top1conf` (that class's confidence score).
4. **Confidence gating**: any prediction below 35% confidence is treated as "unsure" and shown to the user as an explicit warning rather than a definitive diagnosis. This guards against the same fundamental softmax issue discussed in Section 02 — the model always produces some

answer, so a low-confidence threshold is the app's way of saying "I'm not confident enough to tell you this with certainty" instead of silently presenting a shaky guess as fact.

5. The final result is saved into the `predictions` table (`user_id`, `image_path`, `prediction`, `confidence`, `time`, `model_used`) and rendered with a plain-language description and a list of recommended remedies, both pulled from a hand-curated `disease_info` lookup dictionary.

5.3 Dashboard & Analytics

The dashboard aggregates the logged-in user's own prediction history via straightforward SQL grouping (`GROUP BY prediction` for the disease breakdown, `GROUP BY strftime('%Y-%m', time)` for monthly trends) to build:

- A disease-distribution donut chart
- A monthly scan-activity bar chart
- An overall "health score" — the percentage of the user's scans that came back healthy
- A recent-scans list showing the five most recent results

The **PDF export** is worth calling out specifically as a technical decision: rather than the common approach of screenshotting the rendered dashboard DOM (typically via a library like `html2canvas`) and stuffing that image into a PDF, this app draws the report **programmatically with jsPDF** — rectangles, positioned text, and bar heights computed directly from the underlying JSON data. This is more reliable across different browsers and network environments (screenshot-based capture can silently fail behind certain proxies/tunnels), and it gives full, pixel-level control over the exported report's layout and branding, independent of however the live dashboard happens to be styled that day.

5.4 Internationalization

`deep-translator`'s Google Translate backend translates UI strings on the fly, based on whichever language is currently stored in the user's session, with an in-memory cache (`translation_cache`) so identical strings aren't re-translated on every single request — a simple but meaningful performance optimization once you consider how many repeated UI labels (like "Login" or "Dashboard") appear across every page load.

The app supports 11 languages: English plus ten languages widely spoken across India — Hindi, Urdu, Bengali, Telugu, Tamil, Marathi, Gujarati, Punjabi, Malayalam, and Kannada. This is a deliberate accessibility choice reflecting the target user base: farmers who may be far more comfortable diagnosing a crop problem in their own regional language than in English.

5.5 Data Model

```
users (  
  id            INTEGER PRIMARY KEY,  
  username      TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL  
)  
  
predictions (  
  id            INTEGER PRIMARY KEY,  
  user_id       INTEGER → users.id  (foreign key)  
  image_path    TEXT,  
  model_used    TEXT,  
  prediction    TEXT,  
  confidence    REAL,  
  time         TEXT  
)
```

This is a deliberately simple, normalized schema — a clean one-to-many relationship between a user and their prediction history — appropriately minimal for what the app actually needs, rather than over-engineered with tables and relationships that would only matter at much larger scale.

Design Decisions Worth Defending

Decision	Why it's defensible
YOLO11 for classification, not detection	The task is "what disease is on this leaf," not "where exactly on the leaf is it" — a classification head is simpler, faster to train, and matches the dataset's labeling format (whole-image labels, no bounding-box annotations).
Nano model size specifically	Runs acceptably on CPU in production without needing a GPU server — an appropriate tradeoff of some accuracy for dramatically lower deployment cost and complexity, appropriate at this project's scale.
A separate gatekeeper model instead of a 39th "not a plant" class	Training a genuinely robust "other" class would require a huge and highly diverse negative dataset covering everything that isn't a plant — reusing an existing general-purpose model is far cheaper and still catches the obvious, common cases.
SQLite instead of Postgres/MySQL	Removes all database server setup/operations overhead for a single-instance app — explicitly flagged as a limitation the moment real multi-user concurrency becomes a requirement.
Client-side PDF generation	No server-side headless-browser or screenshot dependency required, and it keeps working even behind restrictive network policies or tunnels where screenshot-capture libraries can fail.
On-the-fly translation instead of static locale files	Made it possible to support 11 languages without manually writing and maintaining translations for every UI string by hand; the tradeoff is a runtime network dependency and translation quality that hasn't been human-reviewed.

Known Limitations

Raising limitations proactively — before being asked — is one of the most reliable ways to signal seniority and self-awareness in a technical interview. Here are the honest ones for this project:

BE READY TO DISCUSS THESE OPENLY

- **SQLite doesn't handle concurrent writes well.** Fine for a demo or single user at a time; a real deployment with many simultaneous users would need Postgres or MySQL.
- **The confidence threshold (35%) and gatekeeper keyword list are hand-tuned heuristics,** not learned from data — they could misclassify an unusual-but-genuinely-valid plant photo as invalid, or occasionally let a non-plant image through.
- **No adversarial or robustness testing.** The model hasn't been stress-tested against blurry photos, unusual lighting conditions, or partially visible leaves beyond whatever natural variation already exists in the PlantVillage dataset itself.
- **PlantVillage images are mostly captured under controlled lab conditions** (plain, uniform backgrounds) — a well-documented critique of this dataset in the research literature is that models trained purely on it can generalize poorly to messy, real-world field photos with cluttered backgrounds, variable lighting, and multiple overlapping leaves.
- **Translation quality is unreviewed** — it's machine-translated, not verified by a human speaker, which carries real risk for a farmer-facing tool where a mistranslated remedy instruction could plausibly cause harm.
- **No rate limiting or abuse protection** currently exists on file uploads or login attempts.

Future Improvements

Pairing every limitation with a concrete next step shows forward-thinking, and gives you something substantive to say if asked "what would you do next?"

- Fine-tune on, or ensemble with, a model trained on real-world field images (e.g. the PlantDoc dataset) specifically to close the lab-vs-field generalization gap described above.
- Replace the keyword-based gatekeeper with a proper, learned out-of-distribution detection method — for example energy-based scoring on the classifier's own logits, or a small dedicated binary "is this a leaf" classifier trained specifically for the task rather than repurposing an ImageNet model's incidental label vocabulary.
- Migrate to Postgres plus object storage (an S3-compatible service) to support genuine multi-user concurrency and horizontally scalable image storage.
- Commission human-reviewed translations specifically for the disease-and-remedy text — the highest-stakes content in the app — while leaving auto-translation in place for lower-stakes UI chrome like button labels and navigation.
- Add model explainability, such as a Grad-CAM heatmap overlaid on the uploaded image, so users can visually see *which region* of the leaf drove the prediction — this both builds user trust and gives you, the developer, a debugging tool for spotting when the model is "looking at" the wrong part of the image.

Glossary of Key Terms

A rapid-reference glossary — useful both for last-minute review and as a way to catch yourself if you're ever unsure how precisely to define something mid-interview.

Transfer Learning — Starting model training from weights already learned on a different, usually much larger, dataset (here, ImageNet) rather than from random initialization, then adapting ("fine-tuning") those weights to a new, more specific task. Works because early layers of a vision model tend to learn generic, broadly reusable features (edges, textures, colors) regardless of the exact end task.

Softmax — The final activation function typically used in multi-class classification, which converts a model's raw output scores ("logits") into a probability distribution that always sums to exactly 1 across all classes. This is precisely why a classifier can never output "none of the above" on its own — see Section 02.

Top-1 Accuracy — The percentage of examples where the model's single highest-confidence prediction is the correct class.

Top-5 Accuracy — The percentage of examples where the correct class appears *anywhere* among the model's five highest-confidence predictions — a more forgiving metric, useful when classes can look visually very similar to each other.

Precision — Of everything the model labeled as class X, what fraction actually was class X. High precision means few false positives.

Recall — Of everything that actually was class X, what fraction did the model correctly catch. High recall means few false negatives.

F1-Score — The harmonic mean of precision and recall, giving a single balanced number when you care about both false positives and false negatives roughly equally.

Confusion Matrix — A grid with true classes on one axis and predicted classes on the other, where each cell counts how often that combination occurred. The diagonal represents

correct predictions; everything off the diagonal reveals exactly which classes the model confuses with which others.

Overfitting — When a model performs very well on the data it was trained on but noticeably worse on unseen (validation/test) data, because it has effectively memorized quirks of the training set rather than learning generalizable patterns. Diagnosed by watching training loss keep dropping while validation loss stalls or starts rising.

Automatic Mixed Precision (AMP) — Performing the numerically safe parts of neural network training in 16-bit floating point instead of 32-bit, to reduce memory usage and increase speed on GPUs with hardware support for fast FP16 math, while keeping a few sensitive operations in full 32-bit precision to protect accuracy.

ONNX (Open Neural Network Exchange) — A framework-agnostic file format for trained neural networks, allowing a model trained in one framework (e.g. PyTorch) to be run in a different runtime or environment (e.g. a browser, a mobile app, or a C++ application) without needing the original training framework installed.

Out-of-Distribution (OOD) Detection — The general problem of recognizing when an input doesn't resemble anything a model was trained on, so the model can decline to answer rather than confidently guessing. Central to why this project uses a gatekeeper model.

WSGI (Web Server Gateway Interface) — The standard interface between a Python web application (like this Flask app) and a production-grade web server (like Gunicorn), allowing them to communicate in a framework-agnostic way. Flask's own built-in server implements just enough of this for local development, but isn't built for production traffic.

Salted Password Hashing — Before hashing a password, a random value ("salt") unique to that password is mixed in first. This means identical passwords produce different stored hashes, defeating precomputed lookup-table ("rainbow table") attacks, and it's why `werkzeug.security`'s helper functions are used instead of hashing passwords directly.

Likely Interview Questions & Model Answers

Q: WALK ME THROUGH WHAT HAPPENS WHEN A USER UPLOADS AN IMAGE.

A two-stage inference pipeline: the gatekeeper model first checks the image is plant-like at all, then — only if it passes — the fine-tuned YOLO11 classifier predicts the specific disease class, with a confidence threshold gating any low-certainty result before it's ever shown to the user or written to the database.

Q: WHY YOLO FOR CLASSIFICATION INSTEAD OF A STANDARD CNN LIKE RESNET?

Ultralytics packages YOLO11 with a ready-made classification head and training API that's very fast to fine-tune and export (including straight to ONNX), with sensible defaults already wired in — automatic mixed precision, image caching, built-in augmentation. Practically, this meant substantially less boilerplate than writing a custom PyTorch training loop for what is, at its core, a fairly standard image classification task.

Q: HOW DID YOU VALIDATE THE MODEL WASN'T OVERFITTING?

By tracking training loss against validation loss across epochs, and by examining per-class precision/recall in the classification report rather than relying on overall accuracy alone. Overfitting would show up as the two loss curves diverging, or as strong performance on common classes masking poor recall on rarer ones — exactly what per-class metrics are designed to expose.

Q: WHAT WOULD YOU CHANGE FOR A REAL PRODUCTION DEPLOYMENT VERSUS THIS DEMO?

Swap SQLite for a proper database with concurrent-write support, add rate limiting and stronger input validation, serve through Gunicorn instead of Flask's development server, move uploaded images off local disk onto scalable object storage, and replace the heuristic gatekeeper with a trained, learned out-of-distribution detector.

Q: HOW DO YOU HANDLE A CASE WHERE THE MODEL IS UNSURE?

A hard confidence threshold — 35% — below which the result is explicitly surfaced as "unsure" rather than presented as a confident diagnosis. It's far better for a tool like this to say "I don't know" than to confidently point a farmer toward the wrong treatment.

Q: WHAT IS THE ACTUAL ACCURACY OF YOUR MODEL?

Fill this in with your real numbers from `classification_report_full.csv` / `raw_training_metrics.csv` in your `Research_Output` folder — pull the final epoch's top-1 and top-5 accuracy, and the overall weighted F1-score, before the interview. "Somewhere around X%, let me check the exact figure" is a visibly weaker answer in the room than a specific number you already have memorized.

Q: WHY FLASK INSTEAD OF DJANGO OR FASTAPI?

Flask is deliberately minimal and unopinionated, which suited a project of this size — Django's built-in admin panel, ORM, and migration system would have added complexity this app doesn't need. FastAPI would be a reasonable alternative if the app were API-first (e.g. serving a separate frontend), but this project needed server-rendered HTML pages, which is squarely Flask's strength with Jinja2.

Q: HOW WOULD YOU SCALE THIS TO HANDLE MANY MORE USERS?

The database is the first bottleneck to address (move from SQLite to Postgres for proper concurrency), followed by moving uploaded images to object storage instead of local disk so the app server itself can be horizontally scaled behind a load balancer without needing shared local filesystem access across instances. Model inference could also be split into its own service if it became a bottleneck, so the web tier and the (potentially GPU-backed) inference tier can scale independently.

Final Cheat Sheet — One-Page Recap

Everything on one page for the elevator ride up.

Model

YOLO11n-cls (Ultralytics), transfer-learned from ImageNet weights

Dataset

PlantVillage — 38 classes, 14 crop species

Split

80/20 train/val, stratified per class

Training

15 epochs · 224×224 input · batch 64 · AMP on · Colab T4 GPU

Gatekeeper

MobileNetV2 (ImageNet) · keyword-based plant/non-plant filter

Confidence floor

35% — below this, result shown as "unsure"

Backend

Flask + Flask-Login + SQLite

Frontend

Jinja2 + Tailwind (CDN) + Chart.js

Export formats

.pt (PyTorch/Ultralytics) and .onnx

Languages

11 total — English + 10 Indian regional languages

Deployment

Gunicorn · persistent disk for DB + uploads · ~2GB RAM minimum

Key design pattern

Two-stage cascade: cheap general filter → expensive specialist model

Before the interview: open your Research_Output folder and write your model's actual final top-1 accuracy, top-5 accuracy, and overall F1-score directly onto this printed page in pen. A specific, memorized number is always a stronger answer than "around X%, I'd have to check."